

UPDATE_FLOW

Helix OTA — Update Flow

Overview

This sequence diagram traces the **complete end-to-end update flow** from the moment a device checks for updates through download, verification, installation, reboot, commit, and final success reporting. It covers both the happy path and key error/retry scenarios.

Diagram

sequenceDiagram

autonumber

participant D as 📱 Device
(Helix OTA Client)

participant API as ☁️ OTA Server
(API)

participant Redis as ⚡ Redis

participant PG as 🐘 PostgreSQL

participant MinIO as 📦 MinIO

participant UE as ⚙️ update_engine
(Android A/B)

participant Dash as 🖥️ Dashboard

Note over D,UE: ══ PHASE 1: CHECK FOR UPDATE ══

D->>API: POST /api/v1/check
{device_id, hw_revision,
current_build, oem_version}

API->>Redis: SET device:{id}:heartbeat (TTL 5m)

API->>PG: Query active campaigns matching device profile

PG-->>API: Campaign + artifact metadata

alt Update Available

API-->>D: 200 OK
{update_available: true,
artifact_url, target_build, hash_sha256, signature, size}

else No Update

API-->>D: 200 OK
{update_available: false, next_check:
3600}

Note over D: Return to IDLE state

end

Note over D,UE: ══ PHASE 2: DOWNLOAD ARTIFACT ══

```

D->>API: GET /api/v1/artifacts/{id}/url (request presigned
URL)
API->>MinIO: Generate presigned GET URL (TTL 1h)
MinIO-->>API: Presigned URL
API-->>D: 302 Redirect to presigned MinIO URL

D->>MinIO: GET {presigned-url} (stream artifact)
MinIO-->>D: Artifact binary stream

Note over D: Download progress reported<br/>via POST /api/v1/
report<br/>{state: DOWNLOADING, progress: %}

D->>API: POST /api/v1/report<br/>{state: DOWNLOADING,
progress: 50%}
API->>PG: INSERT device_report (downloading)
API->>Redis: UPDATE device:{id}:state = DOWNLOADING

D->>API: POST /api/v1/report<br/>{state: DOWNLOADING,
progress: 100%}
API->>PG: INSERT device_report (download_complete)
API->>Redis: UPDATE device:{id}:state = DOWNLOADED

Note over D,UE: == PHASE 3: VERIFY ARTIFACT ==

Note over D: Compute SHA-256 of downloaded file
alt Hash Matches
    Note over D: SHA-256 ✓
else Hash Mismatch
    D->>API: POST /api/v1/report<br/>{state: FAILED, error:
HASH_MISMATCH}
    API->>PG: INSERT device_report (failed)
    Note over D: Retry download (up to 3 attempts)
end

Note over D: Verify Ed25519 signature<br/>against trusted
public key
alt Signature Valid
    Note over D: Signature ✓
else Signature Invalid
    D->>API: POST /api/v1/report<br/>{state: FAILED, error:
INVALID_SIGNATURE}
    API->>PG: INSERT device_report (failed)
    Note over D: Halt update, alert operator
end

D->>API: POST /api/v1/report<br/>{state: VERIFYING → VERIFIED}
API->>PG: INSERT device_report (verified)
API->>Redis: UPDATE device:{id}:state = VERIFIED

Note over D,UE: == PHASE 4: INSTALL UPDATE ==

D->>UE: ApplyUpdate(payload_path)
Note over UE: A/B partition: write to<br/>inactive slot

```

```
D->>API: POST /api/v1/report<br/>{state: INSTALLING, progress:
%}
```

```
loop Installation Progress
  D->>UE: GetProgress()
  UE-->>D: {progress: N%}
  D->>API: POST /api/v1/report<br/>{state: INSTALLING,
progress: N%}
end
```

```
UE-->>D: InstallComplete()
D->>API: POST /api/v1/report<br/>{state: INSTALLED}
API->>PG: INSERT device_report (installed)
API->>Redis: UPDATE device:{id}:state = INSTALLED
```

Note over D,UE: == PHASE 5: REBOOT & COMMIT ==

```
D->>UE: SetActiveSlot(new_slot)
UE-->>D: Slot activated
```

```
D->>API: POST /api/v1/report<br/>{state: REBOOTING}
API->>PG: INSERT device_report (rebooting)
API->>Redis: UPDATE device:{id}:state = REBOOTING
```

Note over D: Device reboots into new slot...

Note over D: Device boots successfully on new build
D->>UE: VerifyBootedSlot() == new_slot

```
D->>UE: MarkSlotSuccessful(new_slot)
UE-->>D: Slot marked as successful
```

Note over D,UE: == PHASE 6: REPORT SUCCESS ==

```
D->>API: POST /api/v1/report<br/>{state: SUCCEEDED, new_build:
"xxx"}
API->>PG: INSERT device_report (succeeded)<br/>UPDATE device
SET current_build = new_build
API->>Redis: DELETE device:{id}:state
API->>PG: UPDATE rollout_stats SET succeeded = succeeded + 1
API->>Notifier: Push event
API-->>Dash: Real-time update (device succeeded)
```

Note over D,UE: == ALTERNATE: INSTALL FAILURE ==

```
Note over D,UE: If update_engine fails during install:
D->>API: POST /api/v1/report<br/>{state: FAILED, error:
INSTALL_ERROR, details}
API->>PG: INSERT device_report (failed)<br/>UPDATE
rollout_stats SET failed = failed + 1
Note over D: Device stays on current slot<br/>(A/B fallback
automatic)
```

Flow Summary

Phase	Device Action	Server Action	Duration (typical)
1. Check	POST /check	Query campaigns, generate presigned URL	< 2s
2. Download	GET artifact (streamed)	Log progress, update device state	1–30 min (artifact size)
3. Verify	SHA-256 + Ed25519 check	Log verification result	10–60s
4. Install	Call update_engine (A/B)	Log progress, update state	2–15 min
5. Reboot	Set active slot, reboot	Mark rebooting state	30–90s
6. Commit	Mark slot successful	Final success report, update stats	< 5s

Error Handling

Error	Detection	Recovery
Hash mismatch	Post-download SHA-256 comparison	Retry download (max 3 attempts)
Invalid signature	Ed25519 signature verification	Halt, report failure, alert operator
Install error	update_engine error code	Automatic A/B slot fallback, report failure
Boot failure	Boot verification after reboot	A/B auto-rolls back to previous slot
Network loss	HTTP timeout / connection error	Exponential backoff retry, resume partial download
Server unreachable	All API calls fail	Queue reports locally, retry on connectivity restore